

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS:

Total Marks: 25

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

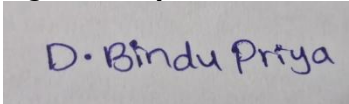
1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Code of Conduct:

I D.BINDU PRIYA(192425162)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



D. Bindu Priya

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1: CNN for Automated Medical Image Classification

1. Suitable CNN Architecture

A suitable CNN can be designed as follows:

Input X-ray → Convolution → Batch Normalization → ReLU → Pooling → Convolution → ReLU → Pooling → Convolution → ReLU → Global Average Pooling → Fully Connected Layer → Softmax

Proposed Architecture

Layer	Configuration	Purpose
Input	$224 \times 224 \times 1$	Grayscale chest X-ray
Conv 1	32 filters, 3×3	Detect edges and basic patterns
Batch Normalization	—	Stabilize and speed up training
ReLU	—	Introduce non-linearity

Layer	Configuration	Purpose
Max Pooling	2×2	Reduce spatial dimensions
Conv 2	64 filters, 3×3	Learn textures and shapes
Batch Normalization + ReLU	—	Improve feature learning
Max Pooling	2×2	Reduce feature-map size
Conv 3	128 filters, 3×3	Detect complex abnormalities
Batch Normalization + ReLU	—	Improve generalization
Max Pooling	2×2	Reduce computation
Conv 4	256 filters, 3×3	Learn high-level disease patterns
Global Average Pooling	—	Convert feature maps to compact representation
Dropout	0.5	Reduce overfitting
Fully Connected	128 neurons	Classification representation
Output	2 neurons + Softmax	Normal / Disease

The model can be trained using **binary cross-entropy loss** and an optimizer such as **Adam**.

2. Motivation and Role of CNN Components

Motivation for Using CNN

CNNs are particularly suitable for X-ray classification because medical images contain spatial patterns that are important for diagnosis. A CNN automatically learns features rather than requiring experts to manually define them.

For example:

Early layers → edges and boundaries → textures → anatomical structures → abnormal regions → disease-related patterns

This makes CNNs effective for detecting visual abnormalities in chest X-rays.

Role of Different Layers

Convolutional layers:

Apply filters to the image and generate feature maps. They identify features such as edges, lines, textures and abnormal structures.

Filters/Kernels:

Small matrices such as 3×3 slide across the image. Different filters learn different visual patterns.

Activation function – ReLU:

Introduces non-linearity:

$$\text{ReLU}(x) = \max(0, x)$$

This allows the network to learn complex relationships.

Pooling layers:

Reduce the spatial dimensions of feature maps while retaining important information. Max pooling keeps the strongest activation in a region.

Fully connected layers:

Combine the learned high-level features and use them for final classification.

Softmax output:

Produces probabilities for the two classes:

- Normal
- Disease-affected

3. Parameter Sharing

One major advantage of CNNs is **parameter sharing**.

In a fully connected network, every neuron is connected to every pixel. For an image of 224×224 pixels:

$$224 \times 224 = 50,176$$

A single neuron would therefore require **50,176 weights**, excluding the bias.

In a CNN, a 3×3 filter has only:

$$3 \times 3 = 9$$

weights for a grayscale image, plus one bias.

The **same filter is reused across the entire image**.

Therefore, instead of learning separate weights for every image location, CNNs learn a small set of filters that can detect the same pattern anywhere.

Benefits

- Fewer trainable parameters
- Lower memory requirements
- Faster training
- Reduced computational complexity
- Ability to detect features regardless of their exact location

For medical images, a learned edge or texture detector can therefore identify that pattern at different positions in the X-ray.

4. Regularization Techniques

Medical datasets can contain thousands of images but may still have limited diversity. A CNN can memorize training images instead of learning general disease characteristics. This is called **overfitting**.

a) Data Augmentation

Training images can be randomly transformed using:

- Small rotations
- Horizontal translations
- Slight scaling
- Cropping
- Brightness adjustments

- Contrast adjustments

This produces additional variations of the training images.

Impact: Improves generalization to X-rays with different positioning, brightness and image quality.

Augmentation must be medically appropriate. For example, transformations that could alter the clinical meaning of an X-ray should be avoided.

b) Dropout

Dropout randomly removes some neurons during training.

For example:

Dropout = 0.5

means approximately 50% of the selected neurons are temporarily ignored during each training step.

Impact: Prevents the network from becoming overly dependent on particular neurons.

c) Batch Normalization

Batch normalization normalizes intermediate activations during training.

Benefits:

- Stabilizes training
- Allows faster convergence
- Helps control internal distribution changes
- Can provide some regularization

Other useful techniques

- Early stopping
- L2 weight regularization
- Learning-rate scheduling
- Transfer learning

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Architecture	Traditional deep CNN	Residual CNN
Depth	8 learned layers	Can have 18, 34, 50, 101+ layers
Feature learning	Good	Excellent
Training difficulty	Relatively easier	Designed to make deep networks easier to train
Vanishing gradients	More susceptible	Reduced through residual connections
Parameter efficiency	Less efficient for modern tasks	Better architectures available
Computational requirement	Lower for original architecture	Higher depending on ResNet version
Medical-image performance	Good baseline	Generally stronger
Complex feature extraction	Limited compared with ResNet	Very strong

Why ResNet is Preferred

ResNet uses residual/skip connections:

$$\text{Output} = F(x) + x$$

Instead of forcing every layer to learn a completely new representation, the network can learn a residual mapping.

This helps information and gradients flow through very deep networks and reduces the **vanishing-gradient problem**.

Final Recommendation

ResNet is more appropriate for the medical image classification system.

The reason is that chest X-rays can contain subtle and complex abnormalities. ResNet can learn progressively more sophisticated features and generally provides stronger classification capability than the older AlexNet architecture.

For practical deployment, a smaller model such as **ResNet-18 or ResNet-34** can provide a good balance between:

Accuracy + feature learning + computational cost.

Therefore:

Recommended architecture: ResNet-18/34 with appropriate medical-image preprocessing, augmentation, batch normalization and dropout.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

1. CNN Architecture for Autonomous Vehicles

The autonomous vehicle needs to recognize:

- Cars
- Pedestrians
- Bicycles
- Traffic signs
- Road obstacles

A suitable CNN pipeline is:

Camera Image → Preprocessing → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Deep Convolution Blocks → Global Pooling/Fully Connected → Output Classes

For example:

Layer	Function
Input	$224 \times 224 \times 3$ RGB image
Conv 1	Detect basic edges
ReLU	Add non-linearity

Layer	Function
Max Pooling	Reduce dimensions
Conv 2	Detect textures and shapes
ReLU	Non-linear feature learning
Max Pooling	Downsampling
Conv 3	Detect object parts
ReLU	Feature transformation
Conv 4+	Learn high-level object representations
Global Average Pooling	Compact feature representation
Fully Connected	Classification
Output	Object classes

For a real autonomous vehicle, the CNN would normally be part of a larger **object-detection system**, rather than simply classifying an entire image.

2. Purpose of CNN Layers and Progressive Feature Learning

Input Layer

Receives RGB camera images, for example:

224×224×3

The image may be normalized before being supplied to the CNN.

Convolution Layer

Convolution applies filters to local regions of the image.

A filter may initially learn:

- Horizontal edges
- Vertical edges
- Corners
- Simple contours

ReLU Activation

ReLU introduces non-linearity:

$\text{ReLU}(x) = \max(0, x)$

It enables the CNN to model complex visual relationships.

Pooling Layer

Pooling reduces spatial dimensions and computational requirements.

For example, 2×2 max pooling selects the strongest feature from each local region.

Fully Connected/Output Layer

The high-level features are combined to determine the object category.

For example:

Object	Predicted Probability
Car	0.82
Bicycle	0.07
Pedestrian	0.08
Sign	0.03

The system would classify the object as a **car** in this example.

Progressive Feature Learning

CNNs learn features hierarchically:

Layer 1: Edges and simple lines

↓

Layer 2: Corners and textures

↓

Layer 3: Wheels, windows, limbs, sign shapes

↓

Deeper layers: Cars, pedestrians, bicycles, traffic signs

↓

Output: Object recognition

This hierarchical learning is one of the major strengths of CNNs.

3. Importance of Parameter Sharing

Parameter sharing means that the same convolutional filter is used at different locations of the image.

For example, a filter that learns to detect a vertical edge can detect that edge:

- On the left side of the road
- In the center
- On the right side
- At different image locations

Instead of having separate parameters for every pixel location, CNNs reuse the same filter.

Advantages

Reduced parameters:

A 3×3 filter requires only 9 weights for one grayscale channel or 27 weights for an RGB image.

Reduced computational and memory requirements:

Fewer parameters need to be stored and trained.

Translation-related feature detection:

The same object feature can be recognized at different spatial locations.

Better generalization:

The network learns reusable visual patterns instead of location-specific patterns.

This is particularly important for autonomous vehicles because objects can appear anywhere within the camera's field of view.

4. Overfitting and Regularization

Autonomous vehicle datasets may contain millions of images, but conditions can still vary significantly. A CNN may overfit to specific roads, cameras, weather or lighting conditions.

Recommended Techniques

1. Data Augmentation

Generate realistic variations using:

- Brightness changes
- Contrast changes
- Rotation
- Scaling
- Cropping
- Horizontal flipping where appropriate
- Weather simulation

This helps the model handle different driving conditions.

2. Dropout

Randomly disables neurons during training, reducing dependence on particular features.

3. Batch Normalization

Normalizes intermediate activations and helps stabilize training.

4. Weight Decay/L2 Regularization

Penalizes excessively large weights and encourages simpler models.

5. Early Stopping

Training is stopped when validation performance stops improving.

6. Transfer Learning

A model pretrained on a large image dataset can be fine-tuned using autonomous-driving images. This can improve training efficiency and performance when the task-specific dataset is limited.

5. AlexNet vs ResNet

Criterion	AlexNet	ResNet
Introduction	Earlier CNN architecture	Modern residual architecture
Depth	Relatively shallow	Much deeper
Architecture	Conventional stacked layers	Residual/skip connections
Feature learning	Good low/mid-level features	Strong hierarchical feature learning
Vanishing gradients	More likely in deeper networks	Greatly reduced

Criterion	AlexNet	ResNet
Training deep networks	More difficult to scale	Easier due to residual connections
Parameters	Relatively high for its capability	Modern ResNet variants provide stronger performance for their size
Accuracy	Good baseline	Usually higher
Computational cost	Lower for original AlexNet	Depends on variant
Complex environments	Less suitable	More suitable
Real-time deployment	Possible	Smaller variants can be suitable

Vanishing Gradient Problem

In very deep conventional networks, gradients can become extremely small during backpropagation.

This makes early layers difficult to train.

ResNet solves this using **skip connections**:

$$y = F(x) + x$$

The shortcut allows information and gradients to pass more directly through the network.

6. Recommended Architecture for Autonomous Vehicles

Recommended: ResNet

ResNet is the better choice for the autonomous vehicle application, **particularly an appropriately sized version such as ResNet-18 or ResNet-34** when computational efficiency is important.

Justification

Accuracy:

ResNet can learn more complex and discriminative features, improving recognition of objects with different shapes and appearances.

Feature learning:

Deep residual networks can learn hierarchical features ranging from edges to complete object structures.

Training:

Skip connections make deeper networks easier to optimize and reduce the vanishing-gradient problem.

Robustness:

The deeper feature representation is useful when objects appear under different lighting, distances and viewing angles.

Computational cost:

Although large ResNet models can be computationally expensive, smaller variants such as ResNet-18 can provide a better accuracy–speed trade-off.

Final Decision

For the autonomous vehicle system, ResNet-18/34 is recommended over AlexNet because it provides stronger feature learning and classification performance while smaller variants can still meet real-time computational requirements.

For an actual autonomous vehicle, however, classification alone is insufficient. The complete system should use an object-detection architecture capable of determining what the object is and where it is located in each camera frame.